

Visualization Library Documentation

A Comparative Guide to Matplotlib and Seaborn

1. Library Overview

Matplotlib and Seaborn are two of the most widely used data visualization libraries in the Python ecosystem. Both are built to help data scientists explore, understand, and communicate patterns in data, but they take different approaches to achieve this.

Matplotlib

Matplotlib is the foundational 2D plotting library for Python, originally released in 2003 and modeled loosely on MATLAB's plotting commands. It provides a low-level, highly flexible interface built around the concepts of a Figure (the overall canvas), Axes (an individual plot area within the figure), and Artists (every visible element drawn on the axes, such as lines, text, and patches). Because it exposes fine-grained control over nearly every visual element, Matplotlib is the go-to choice when a chart needs to be customized precisely or prepared for publication.

Typical use cases: custom scientific plots, publication-quality figures, dashboards that need pixel-level control, and as the rendering engine underneath higher-level libraries like Seaborn and Pandas' plotting API.

Seaborn

Seaborn is a statistical visualization library built directly on top of Matplotlib. It trades some of Matplotlib's low-level control for a much higher-level, dataset-oriented API: instead of manually plotting arrays of x/y values, you generally hand Seaborn a Pandas DataFrame and specify column names for x, y, hue, and other visual attributes. Seaborn also ships with attractive default themes, color palettes, and built-in statistical computations (confidence intervals, regression fits, kernel density estimates), which makes exploratory data analysis considerably faster.

Typical use cases: quick exploratory data analysis, statistical plots that need confidence intervals or aggregations, and visually polished charts with minimal styling code.

2. Graph Types

2.1 Matplotlib

Line Plot

Description:

A line plot connects consecutive data points with a continuous line, making it well suited for showing how a value changes over a continuous variable such as time or an ordered sequence.

Use Case:

Tracking a trend, such as stock price movement over days or temperature over months.

Code Snippet:

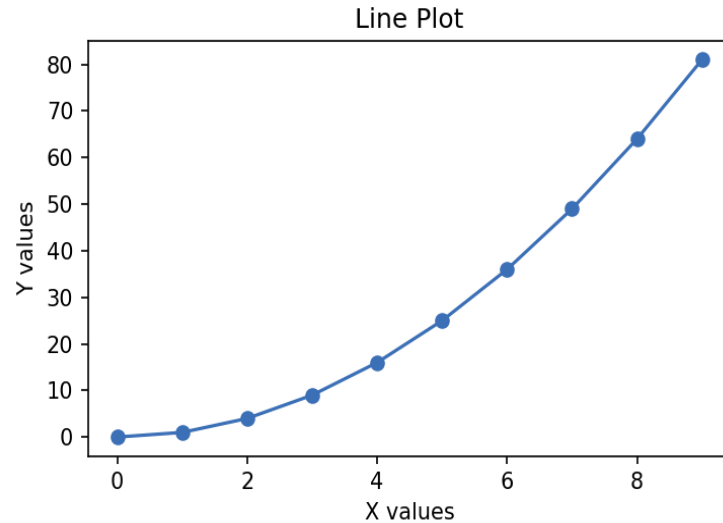
```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(0, 10, 1)
y = x ** 2

plt.plot(x, y, marker='o', color='#3b6fb6')
plt.title('Line Plot')
```

```
plt.xlabel('X values')
plt.ylabel('Y values')
plt.show()
```

Output:



Bar Chart

Description:

A bar chart represents categorical data with rectangular bars, where the length (or height) of each bar is proportional to the value it represents.

Use Case:

Comparing a metric, such as sales, across a small number of categories or groups.

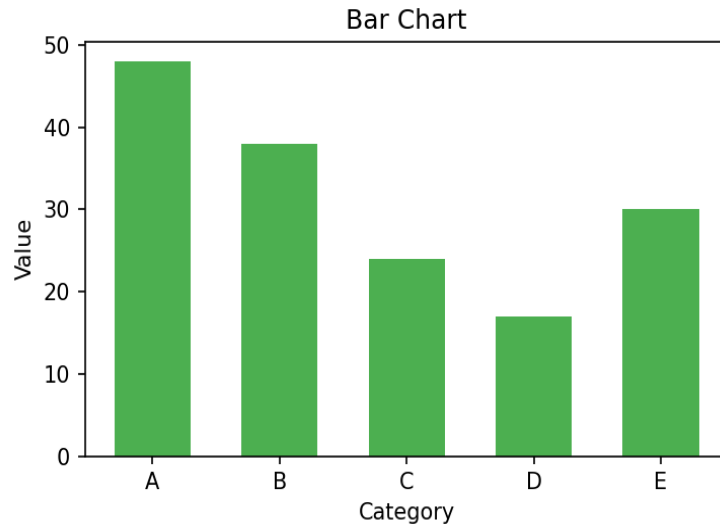
Code Snippet:

```
import matplotlib.pyplot as plt

categories = ['A', 'B', 'C', 'D', 'E']
values = [23, 41, 12, 37, 29]

plt.bar(categories, values, color='#4CAF50', width=0.6)
plt.title('Bar Chart')
plt.xlabel('Category')
plt.ylabel('Value')
plt.show()
```

Output:



Histogram

Description:

A histogram groups continuous numerical data into bins and displays the frequency (count) of observations that fall into each bin.

Use Case:

Understanding the distribution/shape of a numeric variable, such as exam scores or customer ages.

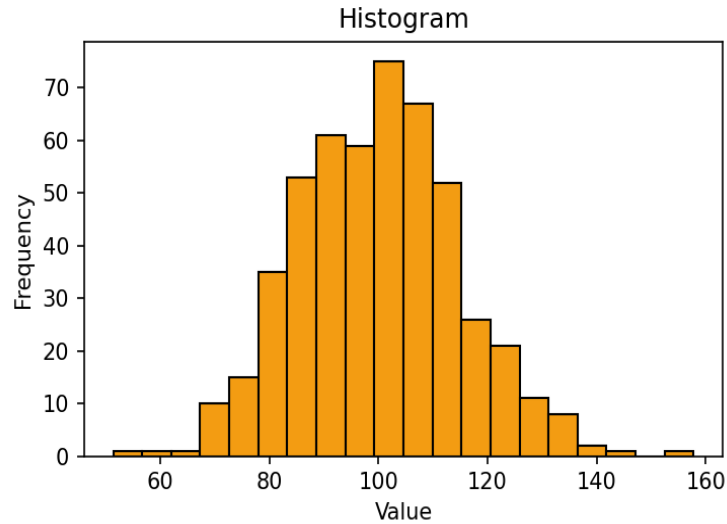
Code Snippet:

```
import matplotlib.pyplot as plt
import numpy as np

data = np.random.normal(loc=100, scale=15, size=500)

plt.hist(data, bins=20, color='#f39c12', edgecolor='black')
plt.title('Histogram')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()
```

Output:



Scatter Plot

Description:

A scatter plot places individual data points on an x-y plane, revealing relationships, clusters, or correlations between two numeric variables.

Use Case:

Checking whether two variables are correlated, e.g. advertising spend vs. revenue.

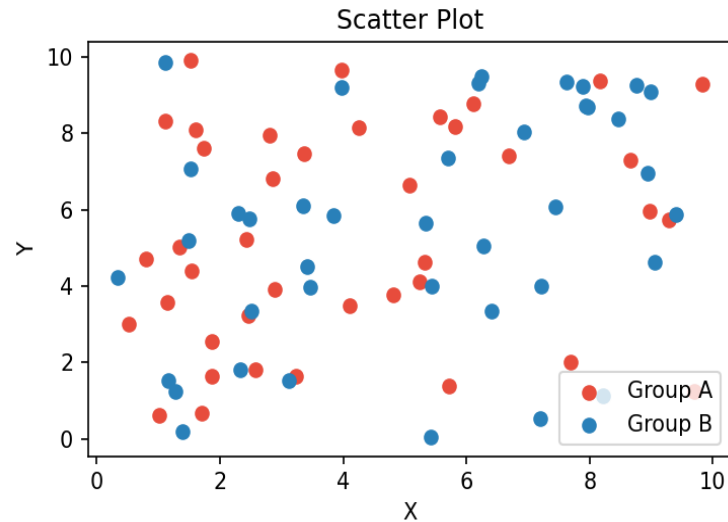
Code Snippet:

```
import matplotlib.pyplot as plt
import numpy as np

x1, y1 = np.random.rand(40) * 10, np.random.rand(40) * 10
x2, y2 = np.random.rand(40) * 10, np.random.rand(40) * 10

plt.scatter(x1, y1, c='#e74c3c', label='Group A')
plt.scatter(x2, y2, c='#2980b9', label='Group B')
plt.title('Scatter Plot')
plt.legend()
plt.show()
```

Output:



Pie Chart

Description:

A pie chart divides a circle into wedges, where each wedge's size is proportional to its share of the whole. It is best used for showing simple part-to-whole relationships.

Use Case:

Showing budget allocation across a small number of categories, such as monthly expenses.

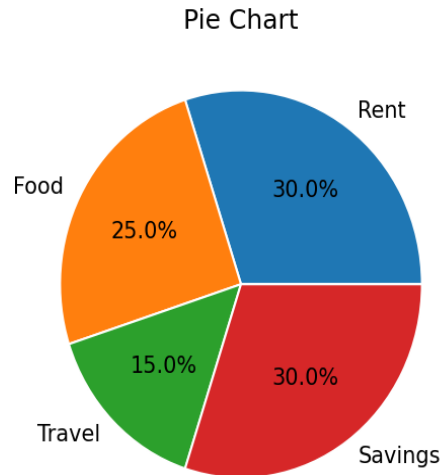
Code Snippet:

```
import matplotlib.pyplot as plt

labels = ['Rent', 'Food', 'Travel', 'Savings']
sizes = [30, 25, 15, 30]

plt.pie(sizes, labels=labels, autopct='%1.1f%%',
        wedgeprops={'linewidth': 1, 'edgecolor': 'white'})
plt.title('Pie Chart')
plt.show()
```

Output:



2.2 Seaborn

Scatter Plot (Relational)

Description:

Seaborn's `scatterplot()` is a relational plot that visualizes the relationship between two numeric variables and can automatically color-code (hue) points by a third categorical variable.

Use Case:

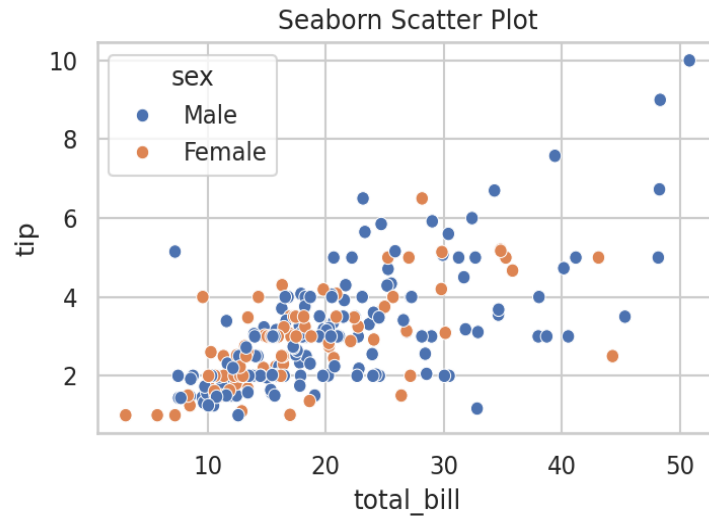
Exploring how two continuous variables relate to each other while also splitting the view by a category, e.g. tip amount vs. bill by sex.

Code Snippet:

```
import seaborn as sns

tips = sns.load_dataset('tips')
sns.scatterplot(data=tips, x='total_bill', y='tip', hue='sex')
```

Output:



Bar Plot (Categorical)

Description:

Seaborn's `barplot()` shows a point estimate (by default, the mean) and confidence interval for a numeric variable, grouped by one or more categorical variables.

Use Case:

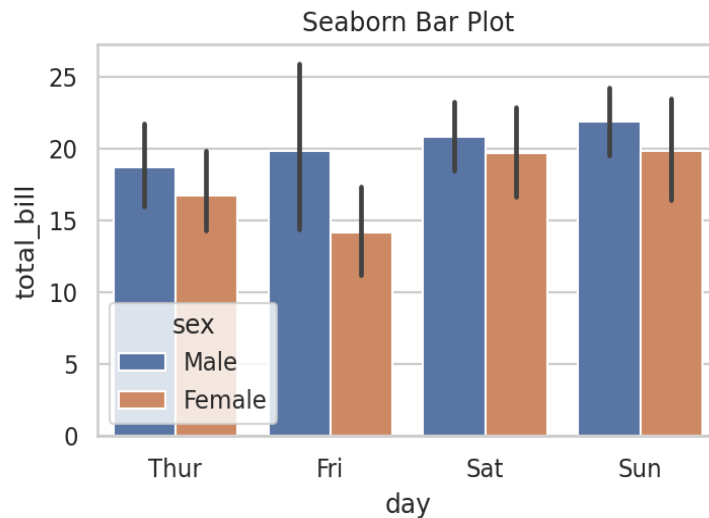
Comparing an average metric across categories with an indication of statistical uncertainty, e.g. average bill by day and sex.

Code Snippet:

```
import seaborn as sns

tips = sns.load_dataset('tips')
sns.barplot(data=tips, x='day', y='total_bill', hue='sex')
```

Output:



Box Plot (Categorical)

Description:

A box plot summarizes a numeric distribution using quartiles: the box spans the interquartile range (IQR), the line inside marks the median, and points beyond the whiskers are flagged as outliers.

Use Case:

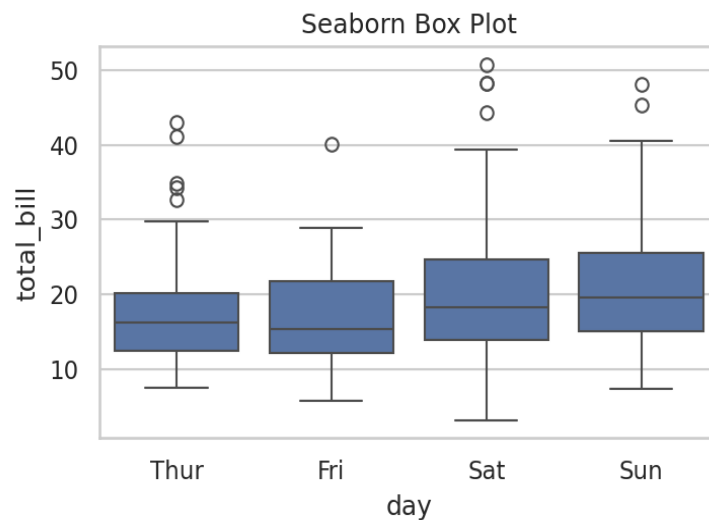
Comparing the spread and outliers of a numeric variable across categories, e.g. bill amount by day of the week.

Code Snippet:

```
import seaborn as sns

tips = sns.load_dataset('tips')
sns.boxplot(data=tips, x='day', y='total_bill')
```

Output:



Histogram + KDE (Distribution)

Description:

Seaborn's histplot() draws a histogram and can optionally overlay a Kernel Density Estimate (KDE) curve, giving both a discrete and smoothed view of a variable's distribution.

Use Case:

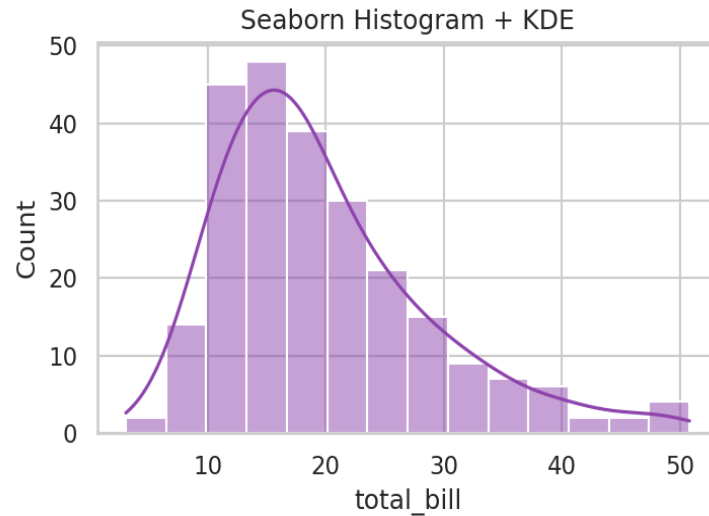
Studying the shape of a distribution (skew, modality) with more nuance than a plain histogram.

Code Snippet:

```
import seaborn as sns

tips = sns.load_dataset('tips')
sns.histplot(data=tips, x='total_bill', kde=True, color='#8e44ad')
```

Output:



Heatmap (Matrix)

Description:

A heatmap encodes a matrix of values (commonly a correlation matrix) as color, with annotations optionally overlaid to show the exact numeric value in each cell.

Use Case:

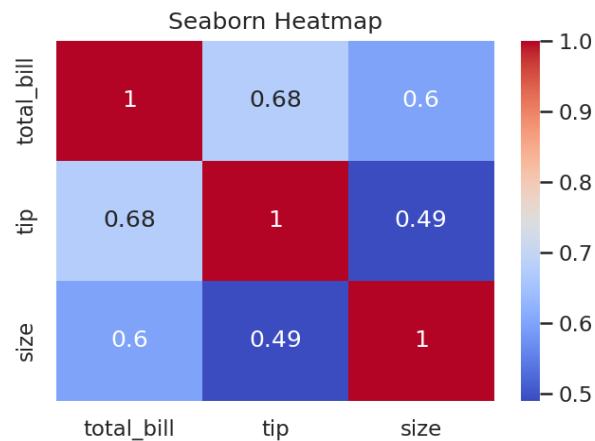
Quickly spotting which numeric variables in a dataset are strongly correlated.

Code Snippet:

```
import seaborn as sns

tips = sns.load_dataset('tips')
corr = tips[['total_bill', 'tip', 'size']].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm')
```

Output:



3. Comparison

Aspect	Matplotlib	Seaborn
Ease of use	Verbose; every element (labels, ticks, legends) is usually set manually.	Concise; works directly with DataFrames and infers most styling automatically.
Customization	Extremely granular — nearly every pixel can be controlled.	Good for common cases, but deep customization often requires dropping down into the underlying Matplotlib objects.
Interactivity	Static by default (interactive backends and widgets exist but require extra setup).	Also static by default — inherits Matplotlib's rendering, so no built-in interactivity.
Performance with large datasets	Handles large datasets reasonably well, though rendering thousands of points can get slow without down-sampling.	Similar performance to Matplotlib since it renders through it; statistical aggregations (e.g. in barplot) add extra computation.

In short: Matplotlib is the better choice when a visualization needs precise, custom control or is destined for a formal report or publication. Seaborn is the better choice when speed of exploration matters more than pixel-level control, especially when the data is already in a tidy Pandas DataFrame and the goal is a statistically informative plot with minimal code. In practice, most data scientists use both together — Seaborn for fast, attractive default plots, and Matplotlib to fine-tune the result when needed.

4. Resources

- Matplotlib: https://matplotlib.org/stable/users/explain/quick_start.html#quick-start
- Seaborn: <https://seaborn.pydata.org/tutorial/introduction.html>